NKI on a Sparse MoE: What Worked, What Didn't, and Why

MLSys 2026 AWS Trainium MoE Kernel Challenge — TEAM-7013, 2nd place

Charles Hong
University of California, Berkeley
charleshong@berkeley.edu

***Abstract*—We describe TEAM-7013's 2nd-place submission to the MLSys 2026 AWS Trainium2/3 MoE Kernel Challenge, which targets end-to-end inference of Qwen3-30B-A3B on a single Trainium3 chip. We wrote roughly a dozen NKI custom-kernel candidates — batched MoE MLPs, RMSNorm, multiple variants of context-encoding (CTE) MoE blockwise matmuls, attention-block fusions, and an LM-head — almost all of which were *bit-exact in isolation* on device yet failed end-to-end accuracy. Two coupled effects are responsible: (i) the Neuron compiler's *Mixed-Precision Accumulation (MPA)* pass fuses $dot \to convert \to silu \to mul$ into a single fp32 pipeline, and any NKI custom call breaks that fusion by forcing a bf16 materialization at the kernel boundary; (ii) even with MPA equalized across both graphs, the compiler's non-MPA SwiGLU lowering rounds at a different sequence of boundaries ($silu(bf16) \to bf16$ followed by $bf16 \cdot bf16 \to bf16$, one rounding event) than a natural NKI implementation does ($silu \to fp32$, $fp32 \cdot fp32 \to fp32$, $cast \to bf16$, two rounding events). We characterize both effects with on-device staged-parity tests and map the small set of *bit-exact cut points* — boundaries where the graph can be split into an NKI call plus a compiler-fused remainder without introducing any numerical drift. The one such cut point we found inside the MoE block — between `silu(g)·u` and the down-projection — is exactly where our shipped CTE MoE NKI kernel pair (*left_gate_up* + *down_scatter*) splits the graph, covering $\sim$99% of CTE MACs and supplying our $1+\rho_{\text{NKI}}$ score multiplier. Most of our score gain came from *speculation*: a co-compiled Qwen3-0.6B fused EAGLE-3 draft (with prompt-lookup self-speculation as a fallback path), delivering bit-exact logits while approximately halving decode latency. We close with concrete advice for next year's NKI MoE optimizer: map fusion boundaries before writing kernels, and treat "bit-exact in isolation" as a necessary but not sufficient correctness condition.**

I. Introduction

The MLSys 2026 AWS Trainium MoE Kernel Challenge asks teams to optimize inference of Qwen3-30B-A3B [1] on a single Trainium3 chip, using NKI [2] custom kernels and graph-level optimizations [3]. Submissions are scored against a compiler-built reference baseline:

$$\text{score} = \text{acc} \cdot \frac{T_b}{T_s} \cdot \frac{\dot{n}_s}{\dot{n}_b} \cdot (1 + \rho_{\text{NKI}}), \tag{1}$$

where T is end-to-end latency, $\dot{n}$ is throughput in tokens/s, and ρ_{NKI} is the fraction of MAC operations executed by NKI custom calls in the scored decoder bucket. Accuracy is binary, gated by *logit_validation* with bf16-typical tolerances against that same baseline graph. Crucially, the baseline is recompiled alongside the submission from the same harness, so its compile-time configuration — bucketing, mixed-precision-accumulation, and other flags exposed via *NeuronConfig* — is part of the design surface: matching the baseline at bit-exactness while moving work into NKI is itself an engineering problem we address in §V.

Qwen3-30B-A3B is a 48-layer Mixture-of-Experts model with hidden size 2048, 128 experts, and top-8 routing per token. At TP=4 the per-rank MoE weight tensors are gate-up $\in \mathbb{R}^{128\times2048\times384}$ and down-projection $\in \mathbb{R}^{128\times192\times2048}$. The sparse expert dispatch and the resulting blockwise matmuls dominate both prefill and decode, so MoE was the natural focus for kernel work.

We describe a six-week effort that produced 26 leaderboard submissions and approximately a dozen NKI kernel candidates. The story arc has three acts:

1) We attempted to NKI-ify the obvious targets: batched MoE MLP, RMSNorm, blockwise CTE MoE matmul. Every candidate failed end-to-end accuracy.
2) We characterized the failure mode — two coupled compiler behaviors that HLO does not surface — with on-device staged tests and an exhaustive boundary search, finding exactly one bit-exact cut point inside the MoE block (between `silu(g)·u` and the down-projection) and cut points at the embedding/LM-head endpoints.
3) Using that boundary map, we shipped a CTE MoE NKI kernel pair that emits bf16 at this cut point and covers $\sim$99% of the CTE bucket's MACs; and we pivoted to graph-level latency wins (a co-compiled Qwen3-0.6B EAGLE-3 fused draft, with prompt-lookup self-speculation as a fallback path) that preserve logits exactly while approximately halving decode latency.

The shipped configuration combines (a) co-compiled Qwen3-0.6B fused EAGLE-3 speculation (`NKI_PLAIN_FUSED_SPEC=1`; prompt-lookup is implemented as a fallback gated off by default), (b) the *_nki_cte_moe_left_right* CTE MoE kernel pair (with `NKI_DISABLE_MPA=1` so the baseline graph compiles identically across the NKI custom-call boundary), and (c) graph-isolated buffer hygiene. We open-source the full submission, every gated-off kernel candidate, the sprint log of every experiment, and the reproducibility scripts under Apache 2.0 [3].

II. System and submission constraints

The contest harness imports the submission file via `importlib.import_module` and runs `python main.py --enable-nki --mode evaluate_all --platform-target trn3` with no environment variables and no extra CLI flags. The submission must therefore be a single Python file with all configuration encoded as module-level defaults. Our submission, `qwen_with_nki.py`, is approximately 11,000 lines and contains every kernel we wrote — shipped or not — gated by `NKI_*` flags whose defaults express the production configuration.

The hardware target is one `trn3pd98.3xlarge` instance, with a single Trainium3 device exposing 4 NeuronCores and 144 GB of HBM. We run on Neuron SDK 2.29 (`neuronx-cc 2.24.5133.0`, `torch-neuronx 2.9.0.2.13.24727`, `nxd-inference 0.9.17334`) with NKI 0.3.0. Tensor parallelism is 4 with logical-NeuronCore (LNC) 2; the compiler is run with `--auto-cast=none` for end-to-end bf16/fp32 fidelity matching the baseline.

The benchmark uses five fixed prompts with per-prompt context and sequence lengths drawn from `prompt_data_trn3.csv`: input lengths $\{18, 126, 43, 10, 402\}$ tokens and total sequence lengths $\{64, 256, 128, 64, 640\}$ tokens, so per-prompt output budgets are $\{46, 130, 85, 54, 238\}$ tokens (the harness sets $\text{max\_new\_tokens} = \text{seq\_len} - \text{input\_len}$). Prompt 4 is a long-context retrieval prompt ("Needle"). Batch size is 1.

III. What we tried first: NKI everywhere

Our first six sprints chased the obvious targets. Each kernel below passed isolated correctness tests on device against the stock torch reference, and each failed end-to-end logit validation.

Batched MoE MLP (*_nki_batched_moe_kernel*). Per-token decode kernel that gathers the eight selected experts' weights, runs gate-up @ (2048, 384), SwiGLU activation, and down-projection @ (192, 2048) inside one *@nki.jit* entry, fusing 16 einsums plus Python accumulation into one custom call. We implemented three rounding schedules to match the compiler's bf16 quantization at every intermediate stage. Standalone max divergence was 7.6×10^{-6} (one bf16 ULP). End-to-end: logit validation fails on token 9 of prompt 0.

RMSNorm (*_nki_rmsnorm_kernel*). Tiled fp32-reduce RMSNorm with weight broadcast. The kernel is bit-identical in isolation to the compiler's PWL-table reference, but the baseline compiler fuses *residual_add* + *rmsnorm* + *matmul* into one schedule that our custom call breaks.

Tail-only MoE kernel (*_nki_moe_tail_only_kernel*). Wraps just the down-projection matmul plus the affinity-weighted scatter-add at the end of each MoE block, leaving gate-up and SwiGLU to the compiler. Standalone bit-exact; end-to-end max-error 0.133.

Per-block CTE MoE (*_nki_cte_moe_block_mlp_kernel*). Single-block kernel called inside an outer Python loop over the prefill's 129 blocks. The graph compiles and loads, but inference takes more than 20 minutes per prompt because the XLA graph contains roughly 12,000 NKI custom-calls (129 blocks $\times$ 48 layers $\times$ 2 entries).

Compiler-assisted fallbacks. We also tried the SDK's *shard_on_intermediate_dynamic_while* and *shard_on_block_dynamic_while* kernels — the former is incompatible (it requires $I_{TP} \geq 2048$ but Qwen3 has $I_{TP} = 192$); the latter compiles, gives 96% NKI MACs, but fails accuracy with max divergence 0.75.

In total, eleven distinct NKI kernels passed isolated correctness and failed end-to-end. The exceptions were the CTE MoE *left_right* pair, which we describe in Section V after first explaining the boundary map (Section IV-A) that motivated their design.

TABLE I
On-device staged parity tests for Qwen3 MoE block components, T=512 tokens, bf16 inputs. Boundaries marked with `xm.mark_step()` defeat compiler fusion in the reference path.

Test	Mismatch %	Max divergence
nc_matmul gate_up vs *torch.matmul*	0 %	0
nc_matmul down_proj vs *torch.matmul*	0 %	0
nl.silu vs *AwsNeuronSilu*	0 %	0
NKI matmul + torch silu (with MPA)	68 %	$7.6\cdot10^{-6}$
Full NKI MoE (matmul + act + matmul)	70 %	$8.6\cdot10^{-6}$
Full NKI MoE, all-fp32 internal	54 %	$7.6\cdot10^{-6}$

IV. The accuracy wall

We characterized the failure mode with a sequence of on-device staged-parity tests. The Qwen3 MoE block computes, per active expert,

$$y = (\sigma(W_g x) \cdot W_u x)\, W_d,$$

where σ is SiLU and the bf16 matmuls are performed on NeuronCore tensor engines. Table I reports mismatch rates and max divergence for staged variants compared against the unfused torch reference (with `xm.mark_step()` between every operation, defeating compiler fusion).

Each individual NKI primitive is bit-exact to its compiler-generated counterpart, yet whenever any NKI custom call sits between two compiler-generated operations, divergence appears at roughly 1 ULP per element. Two distinct compiler behaviors stack to produce this drift, and either alone is enough to bust the contest tolerance:

(i) The MPA fusion wall. Confirmed by HLO dumps before and after the compiler's optimization passes, the *Mixed-Precision Accumulation* (MPA) pass fuses chains like $dot \to convert(\text{bf16}) \to silu \to mul$ into a single fp32 pipeline, eliminating the bf16 round-trip between the matmul output and the activation input. NKI custom calls are opaque to MPA, so any kernel insertion forces the compiler to materialize

TABLE II
FUSION BOUNDARIES IN THE QWEN3 MOE BLOCK.

Boundary	Mismatch %	Verdict
input | gate_up	27 %	lossy
gate_up | silu	64 %	lossy
silu | silu·up	42 %	lossy
silu·up | down+aff+scat	0 %	**free**
LM-head | sampling	0 %	**free**

bf16 at the kernel boundary — introducing exactly the bf16 quantization that MPA would otherwise have removed.

(ii) The SwiGLU rounding-sequence mismatch. Even with MPA disabled, the compiler's non-MPA SwiGLU lowering does $silu(bf16) \to bf16$ followed by $bf16 \cdot bf16 \to bf16$ — one rounding event at the multiply. A natural NKI implementation widens the activation block to fp32 ($silu(fp32)$, $fp32 \cdot fp32$, then a single $cast \to bf16$ at the kernel exit), so it rounds at a different boundary. The PWL table that implements *nl.silu* is bit-exact against *AwsNeuronSilu*; the divergence comes from the surrounding silu+multiply rounding schedule, not from the SiLU function itself. We tested three NKI rounding variants (bf16-each-step, all-fp32, and hybrid) against the compiler's MPA-fused output: all three differed on $\sim$25% of elements at 1 ULP.

The accuracy cost of either effect is small (one ULP per element per layer) but compounds across 48 decoder layers and exceeds the contest tolerance of $(10^{-5}, 0.05)$. We refer to the combination as the *accuracy wall*: the set of compiler behaviors that close silently around an NKI custom call and that no amount of kernel hand-tuning can erase as long as the comparator's baseline graph keeps them.

A. Mapping the bit-exact cut points

To characterize where NKI insertion is *safe*, we exhaustively swept candidate cut points in the MoE block by inserting `xm.mark_step()` at every plausible operator boundary and measuring the divergence introduced. The result, summarized in Table II, is that all boundaries inside the MLP break numerics (mismatch $\geq$ 27%) except one: the boundary between $silu(g) \cdot u$ and $down + aff + scatter$. This is the only bit-exact cut point inside the MoE block. The LM-head is also a cut point, by virtue of sitting at the very end of the graph with no downstream fusion surface, and the embedding lookup is a cut point for the same reason at the front.

"Free" marks the bit-exact cut points: an NKI custom call inserted here introduces no measurable divergence vs. the un-fused reference. "Lossy" boundaries fail because the surrounding computation straddles the MPA fusion edge *and* forces the NKI block to adopt a different silu+multiply rounding schedule than the compiler's lowering would; both effects accumulate past the contest tolerance.

This boundary map became our design rule: an NKI kernel candidate must *output bf16 at one of the bit-exact cut points* to avoid the accuracy wall. The shipped CTE MoE kernel pair (*_nki_cte_moe_left_gate_up_kernel* + *_nki_cte_moe_down_scatter_kernel*, dispatched together via `NKI_CTE_MOE_FULL_MODE=left_right`) is built around it: the first kernel handles gather + gate_up + SwiGLU and emits bf16 at the cut point; the second kernel consumes that bf16 and runs down-projection + affinity-weighted scatter-add. We also generated an LM-head kernel that exploits the trivial cut point at the output of the graph; it survived isolation but does not co-exist with fused speculation's $spec\_len{+}1$ verification shape, so the LM-head kernel ships gated off pending shape parameterization.

V. SHIPPED NKI KERNELS: CTE MOE *left_right*

The shipped NKI MAC contribution comes from the context-encoding (CTE) bucket, where the contest grader's *count_nki_flop_ratio* reads *_tp0_bk0* only. The CTE MoE block is a blockwise matmul: tokens are bucketed by expert assignment into N blocks of size B, and each block runs the standard SwiGLU MLP for one expert.

The mechanism that made this work — after a sprint of authoring NKI MoE kernels that were each bit-exact in isolation but failed end-to-end — was *disabling the MPA compiler pass on both the target and baseline graphs*, plus designing the kernel so its bf16 cast lands on the one bit-exact cut point inside the MoE block. With those two changes, the same NKI custom-call that previously broke accuracy now compiles into a graph whose end-to-end logits agree with the baseline at the contest tolerance. The trick is not that NKI now matches the compiler's MPA-fused output bit-for-bit — it doesn't, since both effects in Section IV are still present locally. The trick is that with MPA disabled *symmetrically* on both sides, the comparator's reference also runs the un-fused, single-bf16-cast schedule, and the residual silu+multiply rounding-sequence drift is small enough at the chosen cut point to fall under tolerance.

We replace the compiler-generated CTE MoE block with two *@nki.jit* kernels co-dispatched via the `NKI_CTE_MOE_FULL_MODE=left_right` mode in `qwen_with_nki.py`:

- *_nki_cte_moe_left_gate_up_kernel*: per-block weight-gather of W_{gu} for the active expert, then $x \cdot W_{gu}$ on the tensor engine, then SiLU and elementwise multiply, all in fp32 internally, with one bf16 cast at the kernel exit. Output shape is (N, B, I) in bf16. The bf16 cast lands at the only bit-exact cut point inside the MoE block (Table II).
- *_nki_cte_moe_down_scatter_kernel*: consumes the bf16 activations, multiplies by the per-token affinities, gathers W_d for the same expert per block, runs $\mathrm{act} \cdot W_d$ on the tensor engine, and scatter-adds back to the output by token position.

Together the two kernels cover roughly 99% of CTE MAC operations.[1] On the leaderboard the measured value is $\rho_{\text{NKI}} =$

[1]The remaining $\sim$1% is the attention QKV and out-projection matmuls ($\sim$0.6%) and the LM-head logits projection ($\sim$0.1%), per Sprint 26.4 of our HLO-level audit on prompt 0.

0.989, consistent across all 25 of our submissions, contributing a $1 + \rho_{\text{NKI}} = 1.989\times$ multiplier in Eq. 1.

Some of the early candidates that fed into this kernel pair were auto-generated and tuned with Autocomp [4], an LLM-driven NKI kernel synthesis system also developed by the author; the shipped kernels were then hand-edited to integrate with NxDI's per-block dispatch and to land the bf16 cast on the bit-exact cut point identified above.

Two compiler-side adjustments make the kernel pair accuracy-safe:

1. Symmetric baseline plumbing. The grader compares our model against a *baseline* model loaded with the stock NxDI config. Because our NKI custom call materializes bf16 between gate_up+SwiGLU and down_proj, the baseline must do the same or accuracy validation fails on the bf16 round-trip difference alone. We wrap the baseline's *get_neuron_config_cls* so that the same *moe_fused_nki_kernel_enabled*, *NKI_CTE_MOE_FULL_MODE* and *NKI_DISABLE_MPA* flags are mirrored on both sides. The point is graph parity, not absence of NKI: as long as both graphs round at the same boundaries, their end-to-end divergence stays under the contest tolerance even though neither matches a hypothetical infinite-precision reference.

2. Disabling MPA. With `--auto-cast=none` and `--disable-mixed-precision-accumulation`, the compiler stops fusing $dot \rightarrow convert(\text{bf16}) \rightarrow silu \rightarrow mul$ in the baseline graph too, so the baseline now runs the same single-bf16-cast SwiGLU schedule that our NKI kernel emits. Combined with the cut-point placement above, end-to-end logits agree at the contest tolerance $(10^{-5}, 0.05)$. Our submission ships with `NKI_DISABLE_MPA=1` as a default; setting it back to `0` re-enables MPA on the baseline only and immediately fails accuracy — effect (i) in Section IV reappears because the two graphs no longer round at the same boundaries.

The TKG (decode) bucket runs the stock NxDI MoE path; we measured the vendor's *moe_fused_nki_kernel* as net-neutral on TKG once speculation is on, and shipped without it (`NKI_MOE_FUSED_TKG=0`) to keep the TKG graph identical to the baseline. The remaining shipped NKI kernel surface is therefore CTE-only.

A. Scope of the kernel work

Across the contest we wrote more than a dozen team-authored *@nki.jit* kernels covering most of the obvious surface area: NKI RMSNorm, NKI embedding lookup, batched-MoE TKG, per-block CTE einsum, LM-head, the CTE MoE *left_right* pair we ultimately shipped, identity-boundary diagnostics, a *down_scatter_kernel_pair*, and several attention-block fusions. `qwen_with_nki.py` contains all of them as a single file, gated by `NKI_*` flags whose defaults express the production configuration. Two of those kernels are live on every forward pass under the shipped defaults: the CTE MoE pair from Section V — *_nki_cte_moe_left_gate_up_kernel* and *_nki_cte_moe_down_scatter_kernel* — which together cover $\rho_{\text{NKI}} = 0.989$ of CTE MACs.

VI. The throughput axis: speculation

The CTE NKI kernel pair earns us a $1.989\times$ MAC multiplier that runs once per prompt, but the prompt's CTE pass is a small fraction of total inference time — decode dominates as soon as the output budget exceeds a few dozen tokens. Our *decode-side* score gain came from a different axis entirely: *speculative decoding* [5], [6]. We implemented two complementary paths, gated by `NKI_PLAIN_FUSED_SPEC` and `NKI_PROMPT_LOOKUP_SPEC`; the shipped default is the fused EAGLE-3 path. The prompt-lookup path is described first because it is the simpler design and the cleaner pedagogical example; it is also live as a fallback in our codebase.

A. Prompt-lookup self-speculation (fallback path)

We implement prompt-lookup candidates [7], [8]: at each decode step, search the entire generated prefix (prompt + generated so far) for the last n tokens of the current sequence. If a match is found at position p, the k tokens that follow at $p+n$ are proposed as candidate next tokens. This is a draft-free "self"-speculation: the proposals come from the prompt itself.

We coexist a *token_generation_model* ($n_{\text{active}} = 1$) and a *speculation_model* ($n_{\text{active}} = spec\_len$). On each step: (a) lookup proposes $spec\_len-1$ candidate tokens; (b) the speculation model verifies all candidates in one forward; (c) we accept the longest matching prefix plus one bonus token from the target's argmax; (d) the KV cache, written for all $spec\_len$ positions, is "rolled back" implicitly because future writes are position-indexed and the next forward's $position\_ids$ point past the accepted prefix. No explicit cache-clear is required.

The emitted logits stream is *identical* to plain greedy TKG — every accepted token is the target's own argmax at the verified position — so logit validation passes at the same tolerance as the baseline.

Hyperparameters that ship: $spec\_len$=5, $ngram$=4, $ngram\_min$=1. Lower $ngram\_min$ means single-token matches still seed candidates; we found $ngram\_min$=1 worth +7.8% over $ngram\_min$=2, because the bonus token is always emitted whether or not any candidate is accepted.

B. Co-compiled Qwen3-0.6B fused draft (shipped path)

Prompt-lookup acceptance varies dramatically across prompts: it is nearly perfect on long-context retrieval (where the model copies spans of source text) but poor on open reasoning (where output is mostly novel tokens). To cover the latter case we co-compile a Qwen3-0.6B dense draft model into the target graph using NxDI's fused-speculation path (*enable_fused_speculation=True*), with the EAGLE-3 wiring [9] and greedy assisted decoding. Both draft and target run in one Neuron graph invocation per step; the draft proposes $spec\_len-1$ tokens, the target verifies, and the acceptance decision lives entirely on-device.

The draft and target share tokenizer and vocabulary (both Qwen3-family, $V = 151{,}936$), so no projection layer is needed. Compile cost is approximately 20–30 minutes one-shot, then cached. Runtime cost per step is one draft forward + one

TABLE III
SPECULATION ACCEPTANCE BY N-GRAM SIZE, FULL-BENCHMARK TRACE. AVG. ACCEPT INCLUDES THE BONUS TOKEN FROM THE TARGET'S ARGMAX. SPEC/TKG BREAK-EVEN IS 1.05.

n	% iters	Avg accept	ms/tok	Profit
0 (fallback)	34 %	1.00	14.57	1.00×
1	46 %	1.34	11.46	1.27×
2	13 %	1.63	9.37	1.56×
3	2 %	1.86	8.26	1.76×
4	5 %	2.49	6.15	2.37×

target forward at n_{active}=*spec_len*. This path *ships* as the default (`NKI_PLAIN_FUSED_SPEC=1`); the prompt-lookup path described above is gated off (`NKI_PROMPT_LOOKUP_SPEC=0`) and kept as a Python-loop fallback that we use to instrument the trace analysis below, since fused-spec hides per-iteration acceptance inside the on-device graph.

C. Trace analysis (prompt-lookup path)

The fused-spec path runs entirely on-device, so its per-iteration acceptance counts are not directly observable. We therefore instrument the prompt-lookup fallback path (`NKI_PROMPT_LOOKUP_SPEC=1`) and report results from a full benchmark run with per-step JSONL telemetry collected. Table III reports per-iteration acceptance broken down by the n-gram size used in the lookup. The break-even acceptance rate is the ratio of speculation-model latency to TKG-model latency, which we measured at $15.33/14.57 \approx 1.05$. Every n$\geq$1 bucket clears that bar; the n=0 row is the non-speculation fallback where we run plain TKG.

The unharvested opportunity is the 34% fallback rate. We tested trivial fallback drafts (repeat-last-token, zero-padding) and they regressed: repeating the last token never produced a single bonus match, so it is pure 0.6 ms/iter overhead. A model-aware fallback (e.g., reusing a previous spec call's unused predictions as a soft cache) is open future work.

VII. OTHER CONTRIBUTIONS AND ABLATIONS

Three small infrastructure changes round out the shipped configuration. (i) **Pre-allocated** *position_ids* **buffer**: we pre-allocate the position-id arange and add the offset in-place each step, instead of recreating `torch.arange(p, p+spec_len)` each iteration. Score delta: +0.27 on the local 4-prompt sum. (ii) **Score-collection bypass**: the Hugging Face `output_scores=True` path appends a per-position copy of logits to a Python list each step; we maintain a pre-allocated flat tensor and write into it directly. Score delta on short prompts (P4): +15%. (iii) **Length-aware lookup strategy**: for very short prompts ($T_0 < 384$ tokens) the n-gram *majority* position in the prefix outperforms the most-recent (*last*) position; for longer prompts *last* wins. We auto-select per-prompt.

Compiler-flag changes did not help. `-O2` on CTE regressed all five prompts by 0.1–0.2 score units; the larger `cc-pipeline-tiling-factor=4` regressed by 1.2%. We kept the compiler at its defaults (`-O1`, tiling factor 2). Table IV summarizes the contribution of each change, measured against the rolling-best at the time of the experiment.

TABLE IV
ABLATION. Δ SCORE IS LOCAL 4-PROMPT TOTAL (P0+P2+P3+P4) VS THE ROLLING-BEST AT THE TIME OF EXPERIMENT. P1 IS EXCLUDED BECAUSE IT IS LOCALLY FLAKY.

Change	Δ score	Status
`NKI_CTE_MOE_FULL_MODE=left_right`	$1+\rho$: 1.0 → 1.989	default
`NKI_DISABLE_MPA=1` (enables above)	accuracy gate	default
Speculation *spec_len*=5 (prompt-lookup)	**+12.46**	default
Buffer hygiene + score-collect bypass	+1.39	default
Pre-allocated position-id arange	+0.27	default
Persistent spec buffers + vectorized commit	neutral	default
Length-aware lookup strategy (auto_t0)	+0.06	default
majority-1 1-gram lookup	+0.39	default
KV-warm fallback	+0.65	default
Co-compiled Qwen3-0.6B fused draft	+1 to +3	default
`NKI_CTE_OPT_LEVEL=-O2`	−0.52	reverted
`NKI_CC_TILING_FACTOR=4`	−0.23	reverted
`NKI_LM_HEAD=1` (fused-spec shape clash)	accuracy fail	gated off
NKI batched-MoE / RMSNorm / per-block CTE kernels	accuracy fail	gated off

VIII. LESSONS FOR NEXT YEAR'S MOE OPTIMIZER

We close with two concrete recommendations.

1. Map fusion boundaries before writing kernels. The single most useful tool we built was an `xm.mark_step()` sweep over candidate cut points in the MoE block. Spending one engineer-day on this would have saved approximately twenty engineer-days of writing NKI kernels we eventually had to discard. An automated "where-would-MPA-fuse" analyzer based on HLO would be a high-leverage SDK contribution.

2. Bit-exact in isolation is not bit-exact end-to-end — unless you also adjust the baseline. Every NKI MoE kernel we wrote was bit-exact in stand-alone tests on device. None survived *end-to-end* logit validation against the unmodified compiler baseline, because two coupled compiler behaviors — MPA fusion of $dot \rightarrow silu \rightarrow mul$ and the compiler's $silu(bf16) \rightarrow bf16 \cdot bf16$ SwiGLU lowering — both differ silently from a natural NKI implementation, and HLO dumps surface neither. Two recipes restore parity: (a) place the kernel's bf16 output on a bit-exact cut point (Table II), and (b) when no such cut point exists, disable the offending compiler pass on *both* the NKI and baseline graphs — e.g., `--disable-mixed-precision-accumulation` mirrored via our symmetric baseline plumbing. Treat in-isolation

correctness as a necessary but not sufficient condition; treat "baseline graph parity" as the additional sufficient condition.

IX. Open source and reproducibility

The full submission, every gated-off kernel, the complete sprint log (approximately 600 lines covering all 39 sprints chronologically), the per-iteration speculation traces, and the reproducibility scripts are released under Apache 2.0 alongside this report. The shortest reproduction path on a Trainium3 instance is:

```
bash scripts/setup.sh
bash scripts/compile.sh
bash scripts/bench.sh
```

`bench.sh` is the verbatim contest-harness invocation (`python main.py --enable-nki --mode evaluate_all --platform-target trn3`) that the leaderboard grader runs.

Acknowledgments

Thanks to the AWS Neuron team for organizing the challenge and providing Trainium3 access.

References

[1] Qwen Team, "Qwen3 technical report," 2025, model card for Qwen3-30B-A3B and Qwen3-0.6B.

[2] AWS Neuron Team, "Neuron kernel interface (nki) documentation," https://awsdocs-neuron.readthedocs-hosted.com/en/latest/general/nki/index.html, 2026.

[3] ——, "Aws trainium2/3 moe kernel challenge," MLSys 2026 Competition Track. https://github.com/aws-neuron/nki-moe, 2026.

[4] C. Hong, S. Bhatia, A. Cheung, and Y. S. Shao, "Autocomp: A powerful and portable code optimizer for tensor accelerators," arXiv:2505.18574, 2025.

[5] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," *ICML*, 2023.

[6] C. Chen, S. Borgeaud, G. Irving *et al.*, "Accelerating large language model decoding with speculative sampling," *arXiv:2302.01318*, 2023.

[7] A. Saxena, "Prompt lookup decoding," https://github.com/apoorvumang/prompt-lookup-decoding, 2023.

[8] Hugging Face, "Hugging face transformers: Promptlookupcandidategenerator," https://github.com/huggingface/transformers, 2024.

[9] Y. Li *et al.*, "Eagle-3: Scaling up inference acceleration of large language models via training-time test," https://arxiv.org/abs/2503.01840, 2025.